AI-ASSISTED CODING

ASSIGNMENT-13.5

NAME:V.SIDDARTHA

HALL:2303A52062

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

PROMPT

Refactor legacy Python code that uses global variables into a cleaner and more testable design. Identify global values shared across functions and pass them through function parameters or a configuration object.

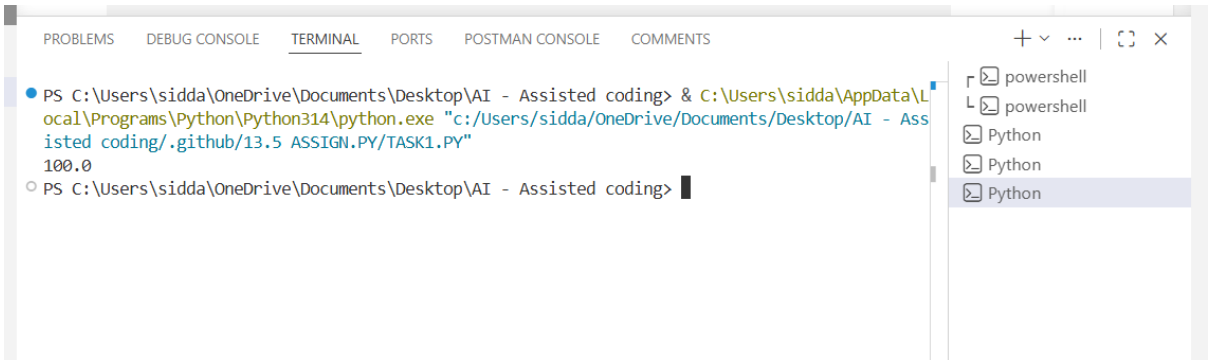
CODE

```
ef calculate_interest(amount, rate):
    return amount * rate
```

```
def main():
    config = {"rate": 0.1}
    print(calculate_interest(1000, config["rate"]))


if __name__ == "__main__":
    main()
```

OUTPUT



Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
```

```
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

PROMPT

Refactor deeply nested if-elif-else logic into a cleaner and more maintainable structure. Improve readability by flattening condition checks using guard-style threshold evaluation.

CODE

```
def get_grade(score):
    thresholds = [
        (90, "Excellent"),
        (75, "Very Good"),
        (60, "Good"),
    ]
    for minimum, label in thresholds:
        if score >= minimum:
            return label
    return "Needs Improvement"

def main():
    score = 78
```

```
    print(get_grade(score))


if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/13.5 ASSIGN.PY/TASK2.PY"
Very Good
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 3 (Refactoring Repeated File Handling Code)

- **Task: Use AI to refactor repeated file open/read/close logic.**
- **Focus Areas:**

o DRY principle

o Context managers

o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.

PROMPT

Refactor repeated open/read/close logic using the DRY principle. Use a reusable function with context managers (with open(...)) so files are safely closed and code is easier to maintain.

CODE

```
def read_and_print(file_path):
    with open(file_path, "r", encoding="utf-8") as file:
        print(file.read())


def main():
    for path in ["data1.txt", "data2.txt"]:
        read_and_print(path)


if __name__ == "__main__":
    main()
```

OUTPUT

Task 4 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**
- **Focus Areas:**

o Time complexity

o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
```

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

PROMPT

Refactor inefficient linear username search by choosing a better data structure. Use set or dictionary lookup for improved time complexity and keep the behavior identical.

CODE

```
def has_access(users, username):
    user_set = set(users)
    return username in user_set

def main():
    users = ["admin", "guest", "editor", "viewer"]
    name = input("Enter username: ")
    print("Access Granted" if has_access(users, name) else "Access Denied")

if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/13.5 ASSIGN.PY/TASK4.PY"
Ent
Acc
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Focus folder in explorer (ctrl + click)

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

PROMPT

Refactor procedural salary-tax calculation code into a class-based design. Apply encapsulation by storing salary and tax rate as object state and expose methods for tax and net salary computation.

CODE

```
class EmployeeSalaryCalculator:
```

```
    def __init__(self, salary, tax_rate=0.2):
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self):
        return self.salary * self.tax_rate

    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()

def main():
    calculator = EmployeeSalaryCalculator(50000)
    print(calculator.calculate_net_salary())

if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/13.5 ASSIGN.PY/TASK5.PY"
40000.0
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 6 (Refactoring for Performance Optimization)

- **Task: Use AI to refactor a performance-heavy loop handling large data.**
- **Focus Areas:**

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

```
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
```

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

PROMPT

Refactor a performance-heavy loop that sums even numbers. Replace iterative logic with an optimized mathematical approach and show a simple timing comparison.

CODE

```
from time import perf_counter

def legacy_sum(limit):
    total = 0
    for i in range(1, limit):
        if i % 2 == 0:
            total += i
    return total

def optimized_sum(limit):
    # Sum of even numbers below limit: 2 + 4 + ... + 2k = k(k + 1)
    k = (limit - 1) // 2
```

```
    return k * (k + 1)


def main():
    limit = 1_000_000


    start = perf_counter()
    legacy_result = legacy_sum(limit)
    legacy_time = perf_counter() - start


    start = perf_counter()
    optimized_result = optimized_sum(limit)
    optimized_time = perf_counter() - start


    print("Legacy:", legacy_result, "Time:", f"{legacy_time:.6f}s")
    print("Optimized:", optimized_result, "Time:", f"{optimized_time:.6f}s")


if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/13.5 ASSIGN.PY/TASK6.PY"
Legacy: 249999500000 Time: 0.039255s
Optimized: 249999500000 Time: 0.000004s
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.

PROMPT

Refactor code that mutates shared global state. Replace side-effect-based functions with functions that return updated values for improved predictability.

CODE

```
def add_item(data, item):-3EW2A
    return data + [item]

def main():
    data = []
    data = add_item(data, 10)
    data = add_item(data, 20)
    print(data)
```

```
if __name__ == "__main__":
    main()
```

OUTPUT

Task 8 (Refactoring Complex Input Validation Logic)

- **Task: Use AI to simplify and modularize complex validation rules.**
- **Focus Areas:**
  - o Readability
  - o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

PROMPT

Refactor nested password validation into modular functions with clear responsibilities. Keep validation readable and easy to test

CODE

```
ef is_long_enough(password):
    return len(password) >= 8

def has_digit(password):
    return any(char.isdigit() for char in password)

def has_uppercase(password):
    return any(char.isupper() for char in password)

def validate_password(password):
    if not is_long_enough(password):
        return False, "Password too short"
    if not has_digit(password):
        return False, "Must contain digit"
    if not has_uppercase(password):
        return False, "Must contain uppercase"
    return True, "Valid Password"

def main():
    password = input("Enter password: ")
    _, message = validate_password(password)
    print(message)

if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/13.5 ASSIGN.PY/TASK8.PY"
Enter password: 1233
Password too short
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

dexing completed. Open Website Generate Commit Message Ln 53, Col 1 (717 selected) Spaces: 4 UTF-8 CRLF